Portline IDP — Extended Technical Report

Intelligent Document Processing and Multi-Shipment Cross-Checking Platform for Customs Compliance

Based on: System Technical Design Document, Release V1.2.0 (Production Spec Verified)

Project	Portline IDP
Document	Extended Technical Report
Version	V1.2.0 (Production Spec Verified)
Repository	github.com/portline-customs/customs-idp-engine
Application	app.portline-customs.internal

1. Executive Summary and System Intent

Portline IDP is an enterprise-grade Intelligent Document Processing (IDP) platform built for international customs brokers, freight forwarders and logistics compliance officers. It automates the ingestion, parsing, verification and audit reconciliation of complex multi-page trade documentation, including Commercial Invoices, Bills of Lading (BL), Packing Lists (PL) and Customs Declarations.

Traditional customs compliance depends on people manually comparing paper and PDF records to find discrepancies in declared line-item values, net and gross weights, currency codes and Harmonized System (HSN) tariff classifications. This work is slow, repetitive and error-prone, and a single missed mismatch can mean penalties, shipment holds or rejected declarations. Portline IDP removes the manual data-entry bottleneck while keeping strict control over data privacy. It does this by running a local, self-contained Gemma 2 (2B Instruction-Tuned) language model inside isolated Docker container infrastructure, so sensitive trade documents never leave the operator's environment.

Core mission. Deliver 99%+ field extraction accuracy across trade documents, execute sub-second multi-document cross-checks, and provide a privacy-first AI Copilot that operates with zero data leakage to external cloud APIs.

Key facts at a glance

Item	Detail
Release	V1.2.0, Production Spec Verified
Repository	github.com/portline-customs/customs-idp-engine
Live application	app.portline-customs.internal (internal network)
Core AI engine	Gemma 2 (2B) local LLM, Vision AI extraction, vector-based HSN search
Backend	FastAPI (ASGI), Pydantic v2, AsyncPG
Data stores	PostgreSQL 16 (with vector support), Redis 7
Frontend	React 18, Vite, Tailwind CSS, Lucide icons
Deployment	Docker Compose with a host-mounted Hugging Face cache

2. Architecture Overview and Component Topology

The system follows a modular, multi-tier microservices topology designed for asynchronous execution, horizontal scaling and local volume caching. It has four logical tiers.

Presentation tier: React Dashboard. The single-page application contains the login and authentication state, the AI Copilot (implemented in `NewSection.jsx`), the Compare and Analyze workspace, and the Document Review Canvas. All backend traffic goes through a standardized fetch client (`api.js`). The interface uses strict flex-bound isolation so that chat and review components do not overlap or distort.

Gateway tier: FastAPI Engine (ASGI). The gateway communicates with the frontend over REST (HTTP/JSON) secured with JWT. It hosts the Security Headers middleware, a CORS regex origin validator, a non-blocking lifespan manager, Pydantic v2 schema validation, a router aggregator mounted at `/api/v1`, and global exception decorators for uniform error handling.

Persistence tier: PostgreSQL 16 and Redis 7. PostgreSQL (accessed through AsyncPG) holds relational metadata and RBAC users, JSONB extracted entities with bounding boxes, and the immutable audit overrides and history. Redis holds sliding-window rate-

limit counters, OTP verification tokens (TTL 600 seconds), the revoked-token blacklist and the session store.

Inference tier: Local Gemma 2 (2B) Service. This service runs PyTorch with Hugging Face Transformers in float32 on CPU or CUDA. It mounts the host's Hugging Face cache and uses an asynchronous thread lock (`_load_lock`) to isolate model loading.

2.1 Technical rationale and trade-off analysis

FastAPI and AsyncIO over synchronous frameworks. Traditional Python frameworks such as Flask or Django block worker threads during I/O operations and database queries. FastAPI uses Python's async/await model through uvicorn and asyncpg, which lets the service handle thousands of concurrent client connections without socket exhaustion. This matters for a platform that waits on database reads, cache lookups and long-running AI work.

Local Gemma 2 (2B) versus external cloud LLMs. External AI APIs such as OpenAI or Google Gemini bring ongoing per-token costs, unpredictable network latency and compliance exposure when handling sensitive trade documentation. Running `google/gemma-2-2b-it` locally guarantees complete data isolation and predictable performance. The trade-off is lower raw model capability than very large hosted models, which the design accepts in exchange for privacy and cost control.

Non-blocking application lifespan. Loading PyTorch weights is CPU-intensive and holds Python's Global Interpreter Lock (GIL). If model loading sits inside FastAPI's synchronous startup sequence, Uvicorn cannot open port 8000, and browsers time out on login. Deferring `init_gemma_model()` to a background thread through `asyncio.to_thread` lets the server become ready immediately while the weights load in the background.

Host-mounted cache volume. The host's Hugging Face cache directory (`${USERPROFILE}/.cache/huggingface`) is mounted into the container at `/root/.cache/huggingface` . The API container therefore reuses the pre-downloaded 2.6 GB model snapshot across restarts and never re-downloads it over the network.

3. Data Modeling and Database Architecture

Storage uses PostgreSQL 16 to combine strict relational schemas for user management and audit logging with flexible JSONB structures for unstructured extraction payloads.

3.1 Core schemas

Table	Key columns and constraints	Purpose and indexing
users	id (UUID PK), email (unique), password_hash, role, is_verified, created_at	RBAC management. Indexed on email for very fast authentication lookups.
shipments	id (UUID PK), shipment_code (unique), supplier_name, status, port_of_entry	Master container for grouped customs files. Indexed on shipment_code and status.
documents	id (UUID PK), shipment_id (FK), doc_type, file_path, page_count, status	Tracks uploaded physical files (Invoice, BL, PL). Foreign-key index on shipment_id.
extracted_fields	id (UUID PK), doc_id (FK), field_name, value, confidence, bbox (JSONB)	Stores key-value extractions, confidence scores and bounding-box coordinates used for UI highlights.
audit_logs	id (UUID PK), shipment_id (FK), field_name, old_value, new_value, user_id (FK)	Immutable record of human compliance overrides, mandatory for regulatory auditability.
hsn_catalog	id (PK), code (6-digit), description, embedding (vector)	Tariff reference database that supports vector similarity search for HSN code suggestions.

3.2 Design observations

The hierarchy is shipment → documents → extracted fields, so every extracted value can be traced back to a physical file and a shipment. The `audit_logs` table records the previous value, the new value and the acting user for each human override, which gives regulators a defensible trail. The `hsn_catalog` table stores description embeddings, so classification suggestions come from semantic similarity rather than exact keyword matches. The `bbox` column keeps text-position coordinates so the interface can link a value to its location on the page.

3.3 JSONB extraction payload

A parsed Commercial Invoice is stored in a structure like this:

```
{
  "document_type": "COMMERCIAL_INVOICE",
  "invoice_number": "INV-2026-8891",
  "currency": "USD",
  "total_amount": 142500.00,
  "line_items": [
    {
      "item_id": 1,
      "description": "High-Precision Industrial Valves",
      "quantity": 500,
      "unit_price": 285.00,
      "hsn_code": "8481.80",
      "confidence": 0.985,
      "bbox": [120, 450, 850, 480]
    }
  ]
}
```

Each line item carries its own confidence score and bounding box. Document-level fields such as the invoice number, currency and total amount sit at the top level. In the example, 500 units at 285.00 equals 142,500.00, which matches the declared total.

4. AI Workflow and Inference Pipeline

The AI engine combines computer-vision OCR for structured entity extraction with the local Gemma 2 (2B) model for interactive Q&A and automated multi-document cross-checking.

4.1 Stage one: rasterization and vision extraction

Uploaded multi-page PDFs are rendered to PNG image arrays at 110 DPI (`AI_RENDER_DPI`). The primary pages, up to `AI_MAX_PAGES` , are passed to the Vision AI model, which extracts key-value fields and tabular line items. Every extracted entity receives a confidence score, and anything below 0.85 (`LOW_CONFIDENCE_THRESHOLD`) is flagged for human review. This human-in-the-loop checkpoint keeps accuracy high without requiring the model to be perfect.

4.2 Stage two: the Gatekeeper cross-checker

After all documents in a shipment package are parsed, the Gatekeeper engine reconciles entities across document types using three rule families:

- **Line-item quantity verification.** Compares the total units declared on the Commercial Invoice with the physical item counts on the Packing List.
- **Weight tolerance verification.** Confirms that the Gross Weight on the Bill of Lading matches the Net Weight plus Tare Weight on the Packing List within a 1.5% margin. The tolerance allows for rounding and measurement differences while still catching material discrepancies.
- **Consignee and origin consistency.** Ensures supplier names, origin countries and buyer details agree across all files.

The Compare and Analyze workspace presents the outcomes through discrepancy scoring matrices and automated summaries.

4.3 Stage three: local Gemma 2 execution

The LLM service (`app/services/llm_service.py`) manages model weights and inference with native PyTorch and Hugging Face Transformers. The core function is:

```
def generate_gemma_response(system_prompt: str, user_prompt: str, max_tokens:
int = 256) -> str:
    global tokenizer, model
    if model is None or tokenizer is None:
        init_gemma_model()
    # Format using Gemma 2's native chat template syntax
    messages = [{"role": "user", "content": f"
{system_prompt}\n\n{user_prompt}"}]
    prompt = tokenizer.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True)
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_tokens,
            temperature=0.2,
            top_p=0.9,
            do_sample=True,
        )
    return tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
skip_special_tokens=True).strip()
```

The function lazily initializes the model on first use, merges the system and user prompts into a single user turn (Gemma's chat template does not use a separate system role), applies the native chat template, and generates with a low temperature (0.2) and top-p of 0.9. The design intends these settings to produce precise, factual answers for customs inquiries. Only the newly generated tokens are decoded, which strips the prompt from the output.

5. UI and UX Engineering Decisions

The frontend is built with React 18 and Vite, styled with Tailwind CSS and Lucide icons. Several screens were redesigned to remove visual overcrowding and layout distortion.

Screen	File	Key decisions and rationale
Login workspace	Login.jsx	Dual-column layout. The left hero section shows feature cards (Document Intelligence, Gatekeeper Verification, Operational Clarity) beside platform metrics. Client-side URL overrides were removed to prevent unauthorized API origin switching.
AI Copilot	NewSection.jsx	Message items use explicit flex bounds (`minHeight: 0`, `flexShrink: 0`, `wordBreak: 'break-word'`) to stop overlapping text layers.
Status banner	Copilot UI	Background model-loading notices are intercepted and shown in a dedicated amber alert banner (`bg-amber-50`), which keeps the chat history clean.
Compare and Analyze	CompareWorkspace.jsx	Side-by-side shipment review with discrepancy scoring matrices and automated summary generation.

The banner decision ties directly to the backend's non-blocking startup. Because the model loads in the background, the first Copilot requests may arrive before weights are ready. Surfacing that state as a banner, rather than as an error message in the conversation, keeps the experience understandable.

6. Security, Privacy and Compliance Architecture

6.1 Encryption

- **In transit.** Communications between the React frontend, the FastAPI gateway and the PostgreSQL and Redis containers are specified to use TLS 1.3.
- **At rest.** Sensitive PII fields and document metadata are encrypted with AES-256 through the Fernet module of the `cryptography` library, driven by the system `AES_ENCRYPTION_KEY`.

6.2 Authentication and session control

- **OAuth2 and JWT.** Short-lived access tokens (15 minutes) are paired with secure HTTP-only refresh tokens (7 days).
- **Redis sliding-window rate limiting.** The `/login` and `/verify-otp` endpoints are limited to 5 requests per 15-minute window per IP address, which blunts brute-force attacks.
- **OTP and revocation.** OTP verification tokens expire after 600 seconds, and revoked tokens are kept in a Redis blacklist so logout and compromise responses take effect immediately.
- **CORS hardening.** `allow_origin_regex` restricts access to authorized developer and production origins.
- **Additional gateway controls.** A Security Headers middleware and Pydantic v2 validation reduce injection and malformed-input risk.

6.3 Privacy posture

Because inference runs locally in an isolated container, no document content or prompt is sent to third-party AI providers. This is the central compliance argument of the platform and the reason the design accepts a smaller model.

7. Technical Debt, Scope Cuts and Roadmap

Domain	MVP scope cut or technical debt	Planned milestone
LLM serving engine	CPU-based PyTorch Transformers execution inside a single container	Deploy a vLLM or TensorRT-LLM server with PagedAttention on NVIDIA A10G GPUs (v1.5)
Customs integration	Manual JSON export of verified document data	Automated Commercial Environment (ACE) EDIFACT XML generation (v2.0)
Document annotation	Textual bounding-box coordinate reporting	Interactive PDF.js canvas with synchronized bounding-box highlights (v1.5)
Real-time updates	Polling HTTP endpoints for status checks	WebSocket connections for real-time OCR progress streaming (v1.8)

The roadmap is sensible in sequence. GPU serving in v1.5 addresses the most significant performance ceiling and ships alongside the PDF.js annotation canvas, which finally uses the stored bounding boxes. Real-time streaming follows in v1.8, and direct customs-system integration is deferred to v2.0, once extraction quality is proven.

8. Deployment and Operational Troubleshooting Playbook

8.1 Local Docker quickstart

```
# 1. Clone repository and configure environment
git clone https://github.com/portline-customs/customs-idp-engine.git
cd customs-idp-engine
cp .env.example .env

# 2. Build and launch containers with host cache volume
docker compose up -d --build

# 3. Monitor non-blocking startup logs
docker compose logs -f api
```

8.2 Troubleshooting scenarios

Symptom	Cause and resolution
CORS login network error	Verify that `allow_origin_regex` in `app/main.py` covers the frontend port in use (for example `http://localhost:5173`).
`IncompleteSnapshotError`	The Hugging Face cache snapshot is corrupted. Remove the model folder inside the container: `docker exec -it customs_idp_backend-api-1 sh -c "rm -rf /root/.cache/huggingface/hub/models--google--gemma-2-2b-it"` and restart so it re-downloads.
PyTorch memory lock on boot	Confirm `init_gemma_model` runs through `asyncio.to_thread` so Uvicorn opens port 8000 immediately.

9. Assessment: Strengths, Risks and Recommendations

This section goes beyond the source document and offers an engineering review of the design as specified.

9.1 Strengths

- Privacy by architecture: local inference removes the largest data-leakage vector.
- Clear separation of concerns across UI, gateway, persistence and inference tiers.
- A built-in human review loop (0.85 confidence threshold) and an immutable audit log, both essential in regulated workflows.
- Operational pragmatism: non-blocking startup and a persisted model cache solve real boot-time and bandwidth problems.
- An honest roadmap that names its own scope cuts.

9.2 Risks and gaps to watch

- **Hallucination claims.** The document says low temperature and top-p sampling "eliminate" hallucinations. These settings reduce randomness but cannot eliminate fabricated output, particularly from a 2B-parameter model. Treat Copilot answers as

assistive and ground them in extracted data, while keeping deterministic Gatekeeper rules as the source of truth for compliance decisions.

- **Accuracy target.** The 99%+ extraction figure is a mission goal. The document does not name the Vision AI model or give measured benchmarks, so validate accuracy on real trade documents and track it per field type.
- **Performance on CPU.** Float32 CPU inference of a 2B model is slow and memory-hungry, and the single container is a scaling bottleneck. The `_load_lock` and background loading help startup but not throughput. The v1.5 GPU serving milestone is therefore important.
- **Cross-checker coverage.** The three rule families cover quantities, weights and party details. Valuation, currency consistency and HSN-versus-description checks are natural extensions, given the schema already stores those fields.
- **Encryption scope.** AES-256 covers PII and metadata. Confirm whether uploaded document files and extracted JSONB payloads are also encrypted, and plan key rotation for `AES_ENCRYPTION_KEY`.
- **Documentation hygiene.** The repository URL is wrapped across two lines in the cover page and could be mistaken for a different path. The quickstart uses `customs-idp-engine`.

9.3 Recommendations

1. Build a labeled evaluation set of real invoices, BLs and packing lists and report extraction accuracy and Gatekeeper precision/recall for each release.
2. Add an automated fallback path so that low-confidence fields and any Copilot answer lacking supporting extracted data are routed to review.
3. Prioritize the GPU serving migration and add load tests that measure Copilot latency under concurrent users.
4. Extend Gatekeeper rules (valuation, currency, HSN consistency) and make tolerances such as the 1.5% weight margin configurable per customs jurisdiction.
5. Add monitoring for rate-limit rejections, model-load state and review-queue depth.

10. Conclusion

Portline IDP is a coherent, privacy-first platform that targets a genuine pain point in customs compliance. Its layered architecture, local Gemma 2 inference, confidence-gated extraction, rule-based Gatekeeper and immutable audit trail work together to reduce

manual effort while keeping a human accountable for final decisions. The documented MVP limits (CPU inference, manual export, textual annotations and polling) are well understood and mapped to concrete milestones. With measured accuracy benchmarks and a stronger inference stack, the platform is well positioned to move from a verified specification to dependable production operation.